

INTERVIEW PREPARATION GUIDE

全栈 / LLM 研发工程实习生 面试准备指南

岗位定向版 | 前端交互 + Python RAG + 微服务 API 集成

南京现场办公岗位 | 基于个人简历、目标 JD 与当前项目代码整理 | 论文部分已排除

王书文

2026 年 8 月 31 日

目录与使用方法

- 第一部分 目标岗位拆解与项目介绍话术
- 第二部分 系统架构、服务边界与 API 集成
- 第三部分 HTML/CSS/JavaScript、Vue/React 与 Node.js
- 第四部分 LangGraph 与 Agent 编排
- 第五部分 RAG 知识库与向量数据流水线
- 第六部分 SSE 流式回答与前端交互
- 第七部分 事务、一致性、取消与并发
- 第八部分 Redis、上下文与异步归档
- 第九部分 Spring Boot、MyBatis 与 MySQL
- 第十部分 Java 并发、虚拟线程与网络调用
- 第十一部分 认证、安全与文件治理
- 第十二部分 Python、FastAPI 与异步编程
- 第十三部分 项目边界、风险话术与简历修改
- 第十四部分 面试流程、答题方法与模拟题
- 第十五部分 14 天复习计划与自测清单
- 附录 高质量资料入口与代码复习路径

建议使用方式：先背熟岗位定向的 90 秒项目介绍，再按“前端交互、JavaScript、Python/FastAPI、RAG 流水线、API 集成、Java 加分项”顺序复习。每个问题都按“为什么这样做、怎么实现、有什么代价、当前还缺什么”四段回答。

内容边界：本文只整理简历中的智能客服项目和相关工程基础，论文与图神经网络部分不在本文范围内。

第一部分 目标岗位拆解与项目介绍话术

1.1 JD 逐项匹配与复习优先级

目标岗位是全栈 / LLM 研发实习生，核心不是传统 Java 后端，而是能把 AI 能力做成用户可用的产品：前端要呈现聊天、状态、引用与失败反馈；Python 要完成 RAG 向量数据流水线和 FastAPI 接口；服务之间要有稳定的 API 契约。你的项目与这三项高度匹配，Java/Spring 则证明你具备更完整的业务系统和微服务基础。

岗位匹配主线：Vue 3 + TypeScript 流式聊天交互、Pinia 状态管理、Axios/Fetch API 集成；Python FastAPI + LangGraph + Chroma 的 RAG 流水线；Java 网关和业务服务作为跨语言微服务集成与工程能力加分项。

- 第一优先级：HTML/CSS/JavaScript、Vue 3 响应式、Pinia、Fetch 流读取、AbortController 和移动端适配。
- 第二优先级：Python、FastAPI、LangChain/LangGraph、文档解析、分块、Embedding、Chroma 和异常降级。
- 第三优先级：REST/SSE 契约、JWT 刷新、幂等键、Problem Details、requestId 和跨服务错误映射。
- Java/Spring/MyBatis/MySQL 内容保留，用于回答项目整体架构、业务数据和微服务集成，但不应抢占开场主线。
- 不要包装成纯算法岗；目标岗位更看重把 LLM 能力落到前端体验和工程链路的能力。
- 不要在没有压测数据时声称高并发、高可用或准确率提升百分比。

1.2 岗位要求到项目证据的映射

岗位要求	项目中的直接证据	面试表达重点
HTML / CSS / JavaScript	Vue 单文件组件、语义化结构、Flex/Grid、媒体查询、Fetch 流读取	不要只背定义，要结合聊天页、消息气泡、移动抽屉和流式解析
Vue / React	Vue 3 Composition API、Pinia、Vue Router、响应式消息对象	Vue 作为主栈；React 回答核心概念和迁移对照，不虚构项目经验

岗位要求	项目中的直接证据	面试表达重点
Node.js	Vite/TypeScript 工具链；项目未用 Node 做后端	掌握事件循环、非阻塞 I/O、Express 中间件与 BFF 场景即可
Python / FastAPI	异步接口、StreamingResponse、Pydantic、依赖注入、LangGraph 运行时	重点讲 I/O 并发、同步库隔离和错误分类
LangChain / RAG	解析、分块、Embedding、Chroma、TopK、引用与降级	能画出离线索引和在线检索两条数据流水线
微服务 API 集成	Axios 拦截器、POST SSE、JWT 刷新、幂等键、requestId、Problem Details	从浏览器请求讲到 Python 事件，再讲回前端状态
代码习惯与自驱	类型定义、service/store 分层、Mock/Remote 双模式、构建检查和错误兜底	准备一个主动定位响应式更新问题并修复的 STAR 故事

1.3 30 秒项目介绍

可直接练习：我做的是一个面向信创售后的全栈智能客服。前端使用 Vue 3、TypeScript 和 Pinia，支持流式回答、阶段状态、引用来源、停止生成与断线终态恢复；Python 使用 FastAPI、LangGraph 和 Chroma 完成 RAG 数据流水线；Java 负责认证、业务数据和 SSE 网关。我重点参与 Agent、向量知识库和接口集成，也能从浏览器一直讲到 Python 检索与数据库终态。

1.4 90 秒项目介绍

推荐主版本：我做的是一个面向信创产品售后的全栈智能客服，用户端和管理端都采用 Vue 3、TypeScript、Vite、Pinia 与 Element Plus。用户提问后，前端先乐观插入消息，通过 POST fetch 接收 SSE 流，并根据 meta、status、delta、citation、done 或 error 更新响应式状态；还支持 AbortController 停止生成、401 单飞刷新、requestId 查询终态和移动端抽屉布局。浏览器只访问 Java 业务服务，Java 负责认证、MySQL 数据和流式网关，再调用 Python FastAPI。Python 侧用 LangGraph 编排 DIRECT/RAG 分支，完成查询改写、文档解析、递归分块、Embedding、Chroma 增量同步和带引用生成。索引以 MySQL 与原文件为真相源，通过 SHA-256 和版本信息避免重复向量化；依赖异常时会明确降级。这个项目让我把前端 AI 交互、Python RAG 流水线和跨语言 API 契约真正串成了一条可追踪链路。

1.5 三分钟展开顺序

- 1. 先讲岗位匹配：这是一个包含 Vue 前端、Python RAG 和跨服务 API 的完整 AI 产品链路。
- 2. 再讲前端体验：流式增量、阶段状态、引用卡片、停止生成、错误恢复和响应式布局。
- 3. 再讲 Python 流水线：FastAPI、LangGraph、解析、分块、Embedding、Chroma 与降级。
- 4. 再讲服务集成：REST + POST SSE、JWT 刷新、幂等键、requestId、错误契约与数据库终态。
- 5. 最后用 Java/Spring 补充业务工程能力，并主动说明尚未实现的混合检索、Reranker 与正式压测。

1.6 系统组件职责速查

组件	主要职责	面试关键词
Vue 用户端	聊天、会话、工单、FAQ 展示; fetch 读取流	Pinia、ReadableStream、AbortController
Vue 管理端	FAQ、维修手册、工单、管理员和审计	权限、CRUD、表单与分页
Nginx	静态资源、HTTPS、API 反向代理、关闭 SSE 缓冲	proxy_buffering、超时、同源
Java 服务	认证、MySQL、FAQ 缓存、工单、手册清单、AI 网关	Spring Boot、事务、MyBatis、SseEmitter
Python Agent	LangGraph、模型调用、RAG、上下文、内部 SSE	FastAPI、async、checkpoint
MySQL	用户、聊天、FAQ、手册、工单等业务真相	事务、索引、状态机、审计
Java Redis	验证码、FAQ Cache-Aside	TTL、缓存一致性、Lua
Agent Redis	checkpoint、会话锁、归档 Stream	分布式锁、消费组、Pending
MongoDB	已完成对话轮次永久归档与冷恢复	幂等写、namespace、排序恢复
Chroma	可删除、可重建的向量派生索引	Embedding、TopK、余弦相似度

第二部分 系统架构、服务边界与 API 集成

2.1 一次聊天请求的完整链路

- 1. 浏览器携带 Access Token，POST 到 Java 的 /api/v1/chat/stream。

2. Java 在短事务中创建会话、聊天请求、用户消息和助手占位消息。
3. Java 立即向浏览器发送 meta，并在虚拟线程中调用 Python 内部 SSE。
4. Python 读取活动角色与 Redis checkpoint，执行 input_guard 和 route_query。
5. DIRECT 直接生成；RAG 依次查询改写、同步索引、检索、返回 citation，再生成。
6. Python 持续发送 status、delta、citation、usage，最后发送 done 或 error。
7. Java 累积最终答案，在事务中提交请求终态、助手消息与引用。
8. 只有数据库提交成功后，Java 才向浏览器发送最终 done。
9. Python 在流结束前把完成轮次写入 Redis Stream，独立消费者再归档 MongoDB。

2.2 为什么拆成 Java 与 Python 两个服务

- Java 更适合承载认证、权限、事务、MyBatis、MySQL 和稳定业务接口。
- Python 的 LangChain、LangGraph、文档加载器和向量生态更成熟，模型迭代成本低。
- 服务分离可以独立升级模型依赖，避免 AI 包与 Java 业务发布周期相互绑定。
- 代价是必须处理内部鉴权、超时、SSE 契约、跨服务错误映射与终态一致性。

2.3 架构高频问答

Q1：为什么浏览器不能直接调用 Python Agent？

Python 不应直接暴露业务权限和数据主键；统一经过 Java 可以复用认证、限流、审计和终态落库，也避免前端知道内部拓扑。

Q2：Java 和 Python 是否共享数据库？

不共享业务写入责任。Java 负责 MySQL 业务表；Python 通过内部清单读取已发布手册，并维护自己的 Redis、MongoDB 和 Chroma。跨服务直接写对方数据会破坏边界。

Q3：为什么 Chroma 不是业务真相源？

向量索引属于派生数据，可能因为模型、切分参数或版本变更而重建。MySQL 手册记录和受管原文件才是可审计的真相。

Q4: 为什么 Java Redis 与 Agent Redis 要分开?

两者的数据模型、故障影响和备份策略不同。FAQ 缓存丢失只影响命中率；checkpoint 丢失会触发上下文恢复。独立部署能隔离容量与运维风险。

Q5: 内部接口为什么不经过公网 Nginx?

内部接口只用于 Java/Python 通信，不需要浏览器访问。Nginx 对 /internal 直接返回 404，可缩小攻击面并防止绕过业务鉴权。

Q6: 这个架构是微服务吗?

更准确地说是 Java 模块化单体加独立 Python AI 服务。不要为了包装而称为复杂微服务体系；当前只有清晰的两个部署单元。

Q7: 如果 Python 不可用会怎样?

Java 将请求标记失败并返回稳定错误；FAQ、工单和后台管理仍可工作。AI 故障不应拖垮全部业务。

Q8: 为什么没有直接引入响应式 WebFlux?

当前内部调用是阻塞式 SSE，Java 21 虚拟线程能以较低改造成本承载 I/O 等待。若未来需要大规模背压与端到端响应式链路，再评估 WebFlux。

Q9: REST 与 SSE 在项目里分别承担什么?

会话、反馈、工单和后台 CRUD 使用普通 REST；模型回答使用 POST SSE。前者关注资源语义和状态码，后者关注事件顺序、增量传输、终态与取消。

Q10: 为什么流式聊天使用 POST，而不是原生 EventSource?

请求需要携带消息体、Bearer Token 和幂等键；EventSource 主要面向 GET 且自定义请求头受限。fetch + ReadableStream 能保留 POST 语义并自行解析 SSE。

Q11: 前后端如何统一错误契约?

普通接口返回稳定的 Problem Details 字段，如 status、code、detail 和 requestId；流式阶段使用 error 事件承载 code、message、retryable。前端不根据自然语言猜错误类型。

Q12: requestId 有什么价值?

它贯穿浏览器、Java 日志、内部 Python 调用和数据库审计，使一次失败能跨服务定位。它用于追踪，不应代替业务幂等键。

Q13: API 如何做向后兼容?

优先新增可选字段，避免修改既有字段语义；事件类型和枚举需要版本策略；破坏性变化进入新路径或协议版本，并在前后端契约测试中验证。

Q14: 开发环境代理与生产反向代理有什么区别?

Vite dev proxy 解决本地联调和同源路径；生产由 Nginx/网关处理 TLS、静态资源、API 路由与流式缓冲配置。开发代理不能当生产安全边界。

第三部分 HTML/CSS/JavaScript、Vue/React 与 Node.js

3.1 项目前端实现证据

- 技术栈：Vue 3、TypeScript、Vite、Pinia、Vue Router、Element Plus；用户端和管理端分开构建。
- 聊天流：普通接口使用 Axios；POST 流式接口使用 fetch + ReadableStream + TextDecoder，自行解析 SSE。
- 状态管理：Pinia setup store 保存会话、消息、生成阶段、AbortController 与当前 requestId。
- 交互状态：meta 创建真实 ID，status 展示阶段，delta 追加文本，citation 展示证据，done/error 收敛终态。
- 可靠性：401 使用共享 refreshPromise 避免并发刷新风暴；请求使用 Idempotency-Key 与 X-Request-Id。
- 响应式修复：流式消息先用 reactive 包装再放入数组，后续原位更新才能稳定触发视图渲染。
- 样式：CSS 变量统一颜色和圆角，Flex/Grid 完成布局，媒体查询和移动端 Drawer 适配窄屏。

- 安全边界：当前回答用文本插值渲染，不直接执行 HTML；若增加 Markdown，需要白名单清洗并限制链接协议。

面试主线：前端部分不要只说“做了几个页面”。应描述为：把不稳定、长耗时、可取消的模型调用，转换成可观察、可恢复、可验证的交互状态机。

3.2 HTML 与 CSS 高频问答

Q1：为什么要使用语义化 HTML？

header、main、section、button、time 等元素能表达内容角色，改善可访问性、键盘操作、SEO 和维护性。语义化不是为了标签数量，而是让结构和行为一致。

Q2：标准盒模型与替代盒模型有什么区别？

content-box 的 width 只计算内容区；border-box 将 padding 和 border 纳入 width。工程中常用全局 border-box，避免组件实际宽度超出预期。

Q3：Flex 与 Grid 如何选择？

Flex 更适合一维排列和内容驱动的对齐，Grid 更适合二维行列布局。聊天页外层可用 Grid/Flex 划分侧栏与主区，消息动作栏适合 Flex。

Q4：什么是 BFC？

块级格式化上下文是独立布局区域，可隔离浮动、阻止外边距折叠并包含浮动。现代布局优先使用 Flex/Grid；需要时可用 display: flow-root 显式创建。

Q5：CSS 优先级如何计算？

大致按内联、ID、类/属性/伪类、元素/伪元素比较，同级后声明覆盖前声明。应通过稳定的组件层级和命名控制，少用 !important。

Q6：position 和层叠上下文怎么解释？

relative 保留文档流并提供定位参照；absolute 脱离普通流；fixed 相对视口；sticky 在阈值内切换。transform、opacity、定位与 z-index 等可能创建新的层叠上下文。

Q7: 如何做响应式布局?

优先弹性尺寸、max-width、Flex/Grid 和内容自然换行，再用少量媒体查询切换布局。项目在窄屏隐藏桌面侧栏并使用 Drawer，消息宽度也按断点调整。

Q8: 如何避免长模型回答撑破布局?

对消息容器设置 min-width: 0，对正文使用 overflow-wrap: anywhere 与 white-space: pre-wrap，并限制消息最大宽度；代码块还需横向滚动。

Q9: scoped CSS 的原理和限制?

构建工具会给元素和选择器追加作用域属性，降低样式串扰；对子组件内部结构通常需要 :deep。它不能替代良好的全局 token 和组件边界。

Q10: 前端可访问性要注意什么?

交互元素使用真实 button/input，提供 aria-label、键盘焦点和状态文本；颜色不能是唯一反馈；流式状态可考虑 aria-live，但要避免每个 token 都被朗读。

3.3 JavaScript / TypeScript 高频问答

Q1: var、let、const 的区别?

var 是函数作用域且存在变量提升；let/const 是块级作用域并有暂时性死区。const 约束的是绑定不可重新赋值，不代表对象内部不可修改。默认优先 const，需要重新赋值再用 let。

Q2: 什么是闭包?

函数与其词法环境的组合，使内部函数在外部函数结束后仍能访问变量。它适合封装状态和工厂函数，但长期持有大对象或 DOM 引用可能造成内存滞留。

Q3: this 如何确定?

普通函数的 this 由调用方式决定：对象调用、显式 bind/call/apply、构造调用或默认绑定；箭头函数没有自己的 this，而是捕获词法环境。

Q4: 原型链是什么?

对象读取自身不存在的属性时，会沿 [[Prototype]] 向上查找，直到 null。class 是原型机制上的语法封装，不是独立继承模型。

Q5: 事件循环、任务和微任务如何运行?

同步栈清空后, 运行时会先清空微任务队列, 再进入下一轮任务和渲染机会。Promise 回调属于微任务, setTimeout 属于任务。大量微任务也会延迟渲染。

Q6: async/await 与 Promise 的关系?

async 函数总是返回 Promise; await 会暂停当前 async 函数并在 Promise settle 后以微任务继续, 不会阻塞整个线程。并行请求应先创建多个 Promise, 再 Promise.all。

Q7: Promise.all、allSettled、race、any 的区别?

all 任一失败即失败; allSettled 等待全部并返回每项结果; race 取第一个 settle; any 取第一个 fulfilled, 全部失败才 AggregateError。

Q8: 浅拷贝和深拷贝有什么区别?

展开语法和 Object.assign 只复制第一层引用; structuredClone 可复制多数结构化数据, 但不复制函数、DOM 节点等。JSON 往返会丢失 undefined、Date 类型等信息。

Q9: 防抖和节流如何选择?

防抖适合输入停止后再搜索; 节流适合滚动或高频进度更新, 保证固定时间最多执行一次。流式 token 渲染可按帧或短时间窗口批量更新, 减少重排。

Q10: 事件委托是什么?

利用事件冒泡, 把多个子元素的监听器放到稳定父节点, 再通过 target/closest 判断来源, 适合动态列表; 对不冒泡事件或复杂隔离边界需谨慎。

Q11: ES Module 与 CommonJS 的区别?

ESM 使用静态 import/export, 便于 tree-shaking, 并支持浏览器原生模块; CommonJS 运行时 require/module.exports, 主要见于传统 Node.js。项目 package.json 的 type: module 表示使用 ESM。

Q12: TypeScript 的价值是什么?

它在编译期约束接口、联合事件和组件参数。项目用判别联合表示 SSE 事件, 使 event 分支能自动缩小 payload 类型; 但运行时输入仍需服务端或 schema 校验。

Q13: unknown 与 any 有什么区别?

any 基本关闭类型检查; unknown 表示类型未知, 使用前必须缩小。处理 catch、JSON 和外部接口时优先 unknown, 再通过守卫或 schema 验证。

Q14: 浏览器存储 Access Token 有什么权衡?

localStorage 易被 XSS 读取; HttpOnly Cookie 不能被脚本读取但需处理 CSRF。项目把 Access Token 放内存、Refresh Token 放 HttpOnly Cookie, 是缩短暴露窗口的一种折中。

3.4 Vue 3 高频问答

Q1: Vue 3 响应式原理是什么?

reactive 使用 Proxy 拦截对象读写, ref 用带 value 的包装对象统一基本类型和对象。副作用运行时收集依赖, 写入后触发相关 effect 和组件更新。

Q2: ref 与 reactive 如何选择?

单值、可替换对象和模板引用常用 ref; 一组关联字段可用 reactive。解构 reactive 会丢失响应式连接, 可用 toRefs, 或避免无意义解构。

Q3: 为什么项目把消息先 reactive 再 push?

messages 是响应式数组, 但继续修改 push 前的普通对象引用, 可能绕过数组内部返回的 Proxy。先 reactive 后保存和更新同一个代理对象, 可保证 delta 原位追加稳定触发视图。

Q4: computed 与 watch 的区别?

computed 用于由状态派生值, 具备缓存且应保持无副作用; watch 用于状态变化后的异步或命令式副作用。会话过滤用 computed, 消息变化后滚动到底部用 watch。

Q5: watch、watchEffect 如何选择?

watch 显式指定来源并可访问新旧值, 控制更精确; watchEffect 自动收集同步执行期间读取的依赖, 适合简单联动, 但依赖边界不如 watch 清楚。

Q6: nextTick 有什么作用?

状态修改后 DOM 更新会被批处理。需要读取更新后的 scrollTop 或操作新节点时 await nextTick; 它不是通用延时工具。

Q7: v-if 与 v-show 的区别?

v-if 真正创建和销毁节点，适合切换少或条件复杂；v-show 只切 display，适合频繁切换但初始渲染成本始终存在。

Q8: v-for 为什么需要稳定 key?

key 帮助 diff 识别节点身份，避免输入状态和组件实例错位。项目先用负数临时 ID，收到 meta/done 后替换为服务端 ID；若 key 改变会重建节点，需要理解这一行为。

Q9: 组件通信有哪些方式?

父传子 props、子传父 emits；跨层可用 provide/inject；跨页面共享业务状态用 Pinia。不要用全局事件总线隐藏数据流。

Q10: Pinia 的作用是什么?

集中管理跨组件状态和动作，同时保留 Vue 响应式与 DevTools 能力。服务层负责 HTTP/协议，store 负责状态转换，组件负责交互呈现，这种边界便于测试和替换。

Q11: Vue 组件的生命周期如何用于资源清理?

onMounted 适合初始化请求和监听；onUnmounted 应移除全局事件、定时器和未完成请求。若聊天组件销毁，应中止读取或明确让后台继续。

Q12: Vue 更新为什么是异步批处理?

同一轮同步代码可能多次修改状态，批处理能合并组件更新，减少重复渲染。读取 DOM 时要等待 nextTick，而业务状态本身通常立即可读。

Q13: 什么是组合式函数?

把可复用的有状态逻辑封装为 useXxx 函数，例如 useStreamingChat 或 useAuthRefresh。它共享逻辑而不是共享状态，除非状态定义在函数外部。

Q14: 如何优化流式回答页面性能?

避免每个字符触发全列表重算，可将 token 合并成小批次；长会话使用虚拟列表；watch 只监听必要字段；Markdown 解析可增量或在批次边界执行。

3.5 React 迁移对照：没有项目经验时怎么回答

Vue 3 概念	React 对应概念	关键差异
ref / reactive	useState / useReducer	Vue 追踪属性依赖；React 通常以新引用触发重新渲染
computed	useMemo	都可缓存派生值；React 依赖数组需要正确维护
watch / watchEffect	useEffect	React effect 面向渲染后的外部同步，并需要清理函数
onMounted / onUnmounted	useEffect(() => ..., [])	开发环境 Strict Mode 可能额外执行以暴露副作用问题
props / emits	props / callback	React 通常把回调函数作为 prop 向下传递
Pinia	Context + reducer / Zustand / Redux	Context 适合低频全局值，复杂高频状态需关注重渲染范围
v-for key	map key	两者都要求稳定身份，不能随意使用数组索引

Q1：React 为什么强调不可变更新？

React 常通过引用变化判断状态是否更新，也便于 memo、时间旅行和并发渲染。直接修改旧对象可能无法触发预期更新或破坏历史快照。

Q2：useEffect 最常见的问题是什么？

依赖遗漏造成陈旧闭包，依赖不稳定造成重复执行，异步请求没有取消造成竞态。effect 应只同步外部系统，能在渲染中计算的值不要放进去。

Q3：受控组件是什么？

表单值由 React state 驱动，通过 value 和 onChange 同步。它便于校验和联动；非受控组件则由 DOM 持有值，通过 ref 读取。

Q4：如何诚实回答 React 经验？

可以说项目主栈是 Vue 3，能用响应式、状态管理和组件化经验映射 React Hooks，并已掌握 state、effect、key 和受控组件；不要声称上线过 React 项目。

3.6 Node.js 与前端工程化

Q1: 项目是否使用了 Node.js 后端?

没有。Node.js 目前用于 Vite、TypeScript 和依赖构建工具链，业务后端是 Java 与 Python。面试中必须明确这一区分。

Q2: Node.js 为什么适合 I/O 密集服务?

JavaScript 主线程执行回调，网络 and 文件等操作由运行时/系统异步处理，完成后进入事件循环。它能以较少线程处理大量等待，但 CPU 密集任务会阻塞主线程。

Q3: Node.js 事件循环要掌握什么?

知道定时器、I/O 回调、poll、check 等阶段，以及 Promise/queueMicrotask 的微任务优先级即可。process.nextTick 队列过多也会饿死 I/O。

Q4: Express/Koa 中间件是什么?

把请求处理拆为按顺序执行的横切逻辑，例如 requestId、日志、鉴权、校验和错误映射。Koa 的 async compose 常被描述为洋葱模型。

Q5: 什么场景会使用 Node BFF?

需要聚合多个后端接口、适配页面模型、处理 SSR 或 WebSocket 时可用 BFF。当前项目 Java 已承担统一入口，额外引入 Node BFF 会增加部署和链路复杂度。

Q6: Vite 为什么开发体验快?

开发时利用浏览器原生 ESM 按需加载，并对依赖预构建；生产构建再进行打包和优化。回答时不要简单说成“完全不打包”。

Q7: 前端环境变量有哪些风险?

打进浏览器 bundle 的变量最终对用户可见，不能放密钥。VITE_ 前缀只决定是否暴露给客户端，不是安全机制。

Q8: 如何保证代码质量?

可采用 TypeScript 严格检查、vue-tsc 构建、ESLint/格式化、组件与 service 分层、关键协议单元测试和 PR 审查。当前仓库能确认的是 TypeScript 构建与类型检查，其他工具不要虚构已配置。

3.7 AI 产品前端专项追问

Q1: 为什么流式接口使用 fetch 而不是普通 Axios 调用?

浏览器 fetch 原生暴露 response.body 的 ReadableStream, 便于逐块读取; 普通 JSON Axios 调用通常等待完整响应。普通 CRUD 仍使用 Axios 拦截器更方便。

Q2: SSE 为什么要维护 buffer?

网络 chunk 边界不等于事件边界, 一个事件可能被拆开, 多个事件也可能合并。解析器必须累积字符串, 按空行切完整事件, 并保留最后半段。

Q3: 停止生成为什么要同时 Abort 和调用取消接口?

AbortController 只停止浏览器读取和本地请求; 模型任务可能仍在服务端运行。取消接口用于表达用户意图并更新后端状态, 两者职责不同。

Q4: 401 刷新为什么要 single-flight?

多个并发请求同时过期时, 只允许一个 refreshPromise 发起刷新, 其余请求等待同一结果, 避免旧 Refresh Token 被并发 rotation 后互相撤销。

Q5: 断网后为什么查询 requestId 终态?

浏览器没收到 done 不代表服务端失败。若服务端已经落库成功, 重新查询可恢复完整答案和引用, 避免把成功请求误显示为失败。

Q6: 如何安全渲染模型 Markdown?

不要直接 v-html 原始输出。先用受控 Markdown 解析器, 再用 sanitizer 白名单清理 HTML, 限制 href 协议, 外链加 rel, 并对代码块和超长内容做资源限制。

Q7: 流式内容会带来哪些性能问题?

高频响应式更新会触发渲染、滚动和 Markdown 解析。可把 chunk 聚合到 requestAnimationFrame 或 30-50ms 批次, 减少全列表 watcher, 并对长会话虚拟化。

Q8: 怎么设计聊天消息状态机?

前端至少区分临时、流式、完成、失败和中断; 服务端事件驱动状态迁移, done/error 是唯一终态。页面切换、重试和反馈操作都应基于明确状态。

第四部分 LangGraph 与 Agent 编排

4.1 当前 workflow

当前图是受控、确定性的客服 workflow，而不是让模型自由决定任意工具的自主 Agent。主路径如下：

- 1. input_guard：规范化本轮用户输入，并清空上一轮临时 RAG 状态。
- 2. route_query：模型只输出 DIRECT 或 RAG；解析失败默认走 RAG。
- 3. rewrite_query：将依赖上文的追问改写为可独立检索的问题。
- 4. sync_manual_index：按 Java 发布清单增量同步 Chroma。
- 5. retrieve：TopK 检索、阈值过滤，并在首个 delta 前发送 citation。
- 6. generate：组合角色 Prompt、对话与证据，裁剪后流式生成。
- 7. output_validate：校验最终消息必须是非空 AIMessage。

关键认知：LangGraph 的价值不是节点越多越好，而是把分支、状态、持久化和失败行为显式化。当前图没有工具自治，因此面试时不要夸大为复杂多智能体系统。

4.2 State、Runtime Context、Checkpoint 与 Store

概念	本项目用途	不能混淆的点
State	messages、route、rewritten_query、retrieved_chunks	会参与图节点传递；临时 RAG 字段每轮重置
Runtime Context	requestId、userId、Prompt、Token 上限、消息 ID	单次运行参数，不写入长期 checkpoint
Checkpoint	按 userId + sessionId 保存线程状态	属于同一会话短期记忆，不是业务消息表
Store	MongoDBStore 保存完成轮次	用于持久数据和冷恢复，不等于向量知识库

4.3 LangGraph 高频问答

Q1: 为什么不用普通 LangChain Chain?

项目存在条件分支、持久状态、恢复、流式事件和节点级降级。LangGraph 能把这些执行语义显式化；若只是一次检索加一次生成，普通 Chain 会更简单。

Q2: MessagesState 的 add_messages reducer 有什么作用?

新消息 ID 会追加，相同 ID 会替换，便于 checkpoint 合并。项目为每次真实运行生成带 UUID 的消息前缀，避免开发环境重建 MySQL 后 requestId 与旧 checkpoint 冲突。

Q3: 为什么每轮要重置 retrieved_chunks?

检索切片只属于本轮问题；如果残留到下一轮，生成节点可能引用错误证据，也会让 checkpoint 不必要地膨胀。

Q4: 路由模型失败为什么默认 RAG?

客服问题可能涉及维修手册事实。默认检索虽然增加成本，但比在企业知识问题上直接生成更保守。

Q5: 查询改写失败怎么办?

查询改写是增强步骤，不是回答的硬依赖；失败时回退原问题继续检索。

Q6: 为什么生成 Prompt 不写回 State?

本轮拼接的检索证据和角色说明是运行输入，不应作为历史消息永久累积，否则会造成上下文污染与存储膨胀。

Q7: 为什么需要 output_validate?

流式模型可能异常结束或返回空内容。终点校验可以阻止系统把空回答当成成功终态。

Q8: 为什么需要 trim_messages?

长对话会超过模型上下文。项目按最后消息优先裁剪，并确认本轮用户消息仍存在；如果本轮问题本身超限，则明确失败。

Q9: checkpoint 和聊天消息表有何区别?

checkpoint 保存图执行需要的状态；MySQL 保存面向用户展示、审计和业务查询的消息。两者生命周期与格式不同。

Q10: 如果让模型自由调用检索工具会更好吗?

不一定。当前检索是确定性节点，便于保证手册问题一定经过同步与引用。工具自治更灵活，但更难评测、限权和复现。

Q11: 角色文件为什么每次请求读取?

可以让下一次运行立即采用新角色而无需重启；已经开始的流仍持有角色快照，避免回答中途 Prompt 变化。

Q12: 如何让图更适合生产?

补充节点级指标、超时预算、重试边界、checkpoint 保留策略、评测集、Prompt 版本和多实例取消状态。

第五部分 RAG 知识库与向量数据流水线

5.1 索引链路

1. 管理员通过 Java 上传 PDF、DOCX、TXT 或 MD。Java 生成安全 objectKey，计算 SHA-256，并把文件元数据写入 MySQL。
2. Python 每次 RAG 请求先读取 Java 的已发布手册清单，不扫描目录猜测哪些文件有效。
3. Python 比较 documentId 对应的 fingerprint。新增或版本变化的文档删除旧切片，再重新解析和向量化。
4. PDF 使用 PyPDFLoader，DOCX 使用 Docx2txtLoader，文本类文件使用 TextLoader。
5. RecursiveCharacterTextSplitter 默认使用约 800 字符切片、120 字符重叠。
6. 切片写入标题、documentId、来源定位、页码、fingerprint 等元数据。
7. Embedding 使用 text-embedding-v3、1024 维；Chroma 使用 cosine 空间持久化。
8. 查询时取 Top 5，再用 0.35 相关度阈值过滤，符合要求的结果先发送引用，再用于生成。

5.2 为什么要做增量同步

如果每次请求都重建全部索引，Embedding 成本、延迟和接口限额都会迅速增加；如果只在上传时让 Java 与 Python 双写，又会产生跨服务一致性问题。当前方案让 MySQL 与原文件保持唯一真相，Chroma 只保存可重建派生数据，通过版本指纹实现惰性增量同步。

面试亮点： 清单拉取失败时，本轮禁止继续查询可能过期的 Chroma，而不是悄悄使用旧索引。这体现了对知识新鲜度和错误答案风险的取舍。

5.3 RAG 高频问答

Q1: RAG 解决什么问题？

把外部知识在推理时检索进上下文，提高企业知识问答的可追溯性和知识更新速度；它不能保证绝对正确，仍需要检索和生成评测。

Q2: 为什么不能说当前是“智能分块”？

代码使用固定参数的 RecursiveCharacterTextSplitter，属于递归字符分块。真正的语义分块通常会依据句子、标题结构或 Embedding 断点决策。

Q3: chunk 越小越好吗？

太小会丢失上下文并增加切片数；太大会稀释语义、增加 Token 和召回噪声。应结合文档结构和评测数据调整。

Q4: 为什么需要 overlap？

避免关键信息恰好落在边界两侧。但 overlap 过大会制造重复结果、增加存储和上下文占用。

Q5: 为什么要保存页码和来源定位？

用于引用展示、问题排查和人工核验。只返回文本而没有来源，会降低企业客服回答的可信度。

Q6: Embedding 是什么？

把文本映射到固定维度的稠密向量，使语义相近文本在向量空间中距离更近。Embedding 维度、模型和距离函数必须匹配。

Q7: 余弦相似度关注什么?

主要比较向量方向，弱化长度影响，常用于文本语义相似度。要注意不同库可能返回 distance 或 relevance score，阈值含义不能想当然。

Q8: TopK 与阈值如何选择?

TopK 决定候选数量，阈值控制最低相关性。当前 5 和 0.35 是初始参数，应以 Recall@K、引用正确率、回答忠实度、延迟和 Token 成本联合调参。

Q9: 为什么需要查询改写?

“它怎么安装”等追问脱离历史后无法独立检索。改写会补全产品、型号、系统版本和故障现象。

Q10: 为什么不直接把整个文档发给模型?

文档可能超上下文、成本高且噪声多。检索只选择相关切片，但会引入召回遗漏，因此需要评测。

Q11: 如何减少幻觉?

要求手册事实只能依据检索证据；证据不足时明确说明；引用先于正文返回；建立无答案样本评测拒答能力。

Q12: 当前为什么没有 BM25?

当前实现是稠密语义检索，工程简单但对型号、错误码等精确词可能不够敏感。可演进为 BM25 + 向量混合检索，再用 Reranker 排序。

Q13: Reranker 有什么价值?

先宽召回，再由更强的交叉编码模型对 query-document 对进行精排，通常提高前几条相关性，但增加延迟和推理成本。

Q14: 索引模型更换后怎么办?

Embedding 模型或维度变化意味着旧向量不可直接复用，应创建新集合或全量重建，并保留切换和回滚策略。

Q15: 如何做 RAG 评测?

拆成检索与生成两层：检索评 Recall@K、MRR、命中率；生成评忠实度、答案相关性、引用正确率、拒答率，再观察 P95 延迟与成本。

Q16：文档解析失败为什么可以跳过单个文件？

一个损坏文件不应阻断所有已发布手册；记录文档 ID 与错误并让后续同步重试。但如果 Embedding 或 Chroma 整体失败，则应让本轮检索降级。

Q17：什么是数据泄漏风险？

检索必须考虑用户权限和文档可见性；当前维修手册按发布状态过滤，若未来有部门或租户隔离，需要在 metadata 查询中强制权限过滤。

Q18：为什么文档清单不传绝对路径？

绝对路径会暴露服务器目录并增加路径注入风险。Java 只传受管 objectKey，Python 再在固定根目录下解析并验证父路径。

第六部分 SSE 流式回答与前端交互

6.1 事件序列

浏览器侧协议按序号检查事件，终态后禁止继续出现事件。主要类型如下：

事件	用途	关键约束
meta	返回 requestId、sessionId 和消息 ID	应最先发送，便于后续恢复
status	展示 safety、intent、retrieval、generation 阶段	只表示进度，不代表成功
citation	返回来源、摘要、页码与分数	RAG 命中时在首个 delta 前发送
delta	增量文本	前端按序拼接，不能假设网络分块等于事件边界
usage	模型与 Token 使用量	可选，不作为业务终态
done	成功终态与最终 messageld	Java 数据库提交后才能发送
error	稳定错误码、消息和 retryable	与 done 互斥

6.2 高频问答

Q1：SSE 与 WebSocket 的区别？

SSE 基于 HTTP，主要是服务端到客户端单向推送，文本协议简单并容易穿过代理；WebSocket 是双向长连接，适合高频交互。客服生成主要是单向增量输出，SSE 足够。

Q2: 为什么前端不用 EventSource?

EventSource 主要发起 GET，难以直接携带 POST JSON 请求体。本项目需要发送 sessionId、message 和 Authorization，因此使用 fetch + ReadableStream。

Q3: 为什么要自己维护 buffer?

网络 chunk 可能在任意位置切分，一个 SSE 事件可能跨多个 chunk，多个事件也可能在同一 chunk。解析器必须累计字符串并按空行分隔事件块。

Q4: 为什么要检查 sequence?

检测重复、乱序或终态后的非法事件，避免前端静默拼出错误回答。

Q5: Nginx 为什么要关闭 proxy_buffering?

默认缓冲可能把多个 delta 累积到响应结束才发送，数据库虽然正常，用户却看不到实时输出。

Q6: 为什么禁用 gzip?

压缩层也可能产生缓冲，影响小片段及时刷新；流式路径通常单独关闭压缩和缓存。

Q7: 浏览器断线是否等于取消?

不等于。网络闪断时服务端可以继续生成并落库，前端再根据 requestId 查询终态。只有用户明确点击停止，才调用取消接口。

Q8: AbortController 做了什么?

它中止浏览器 fetch 读取；还需要单独调用 Java 取消接口，才能把数据库状态和 Python 运行一并取消。

Q9: 为什么 Java 收到 Python done 后不直接转发?

Python 不知道 MySQL 消息主键和业务事务是否成功。Java 必须先落库，取得最终消息 ID，再生成浏览器 done。

Q10: 流意外断开怎么判断?

如果既没有 done 也没有 error，应识别为协议中断而不是成功，并把请求置为失败或允许前端查询恢复。

Q11: SSE 能自动重连吗?

EventSource 有内置重连; fetch 自解析没有。当前策略是以 requestId 查询最终结果。若实现续传, 还需 Last-Event-ID、事件缓存和幂等序列。

Q12: SseEmitter 与 Python StreamingResponse 分别做什么?

Java SseEmitter 面向浏览器输出; Python StreamingResponse 把异步生成器产生的内部事件流给 Java。二者之间还有 JDK HttpClient 解析和事件转换。

第七部分 事务、一致性、取消与并发

7.1 请求状态机

聊天请求主要状态为 ACCEPTED、RUNNING、SUCCEEDED、FAILED 和 CANCELLED。终态不能被后到事件覆盖, 依靠带状态条件的 SQL 更新, 而不是只在 Java 内存中判断。

- ACCEPTED → RUNNING: 只有后台虚拟线程真正开始时更新。
- RUNNING → SUCCEEDED: 回答、引用与请求终态在一个事务中完成。
- ACCEPTED/RUNNING → CANCELLED: 用户取消时先落数据库终态。
- ACCEPTED/RUNNING → FAILED: 内部流中断或模型错误时更新。
- 任何终态再次接收成功、失败或取消更新时, 条件更新影响行数为 0。

7.2 为什么采用短事务

初始化事务只创建会话、请求和两条消息; 随后离开事务调用 Python。流结束后再开启终态事务。这样既避免留下半条请求, 也不会模型推理期间长期占用数据库连接和锁。

7.3 高频问答

Q1: 取消与成功同时发生, 如何保证取消不被覆盖?

取消 SQL 先把 RUNNING 改为 CANCELLED; 成功 SQL 只更新 status=RUNNING 的行。若取消先提交, 成功影响行数为 0, 回答和引用不会写入。

Q2: 为什么数据库状态条件比 `synchronized` 更可靠?

`synchronized` 只保护单进程内线程, 无法覆盖多实例和数据库外的并发。带前置状态的原子 `UPDATE` 把竞争判断放到共享真相源。

Q3: 什么是幂等?

同一逻辑请求重复执行一次或多次, 最终业务结果相同。幂等不等于接口永不报错, 也不等于没有重复网络调用。

Q4: 当前 Chat 是否实现了通用 `Idempotency-Key`?

没有。`request_hash` 当前只用于审计, `api_idempotency_record` Mapper 也未参与 Chat。前端虽然发送 Header, 但不能把它描述成后端已实现的通用幂等。

Q5: 取消接口为什么可以称为幂等?

第一次把活动请求改为 `CANCELLED`; 后续取消看到请求已终止, 不再产生新副作用, 并返回现有状态。

Q6: 为什么外部调用不能放在数据库事务里?

模型调用时间长且不可控, 会占用连接、扩大锁范围并增加回滚成本。外部调用还无法参与本地 `ACID` 事务。

Q7: 如果回答落库成功但浏览器没收到 `done` 怎么办?

浏览器已经持有 `requestId`, 可调用结果查询接口恢复 `SUCCEEDED` 与最终答案。这是把业务终态和传输连接解耦。

Q8: 如果浏览器收到部分 `delta` 后服务端失败怎么办?

前端将占位消息标记 `FAILED`; 数据库保存失败终态而不是把部分文本当作完整回答。是否保留部分内容属于产品决策。

Q9: 引用和回答为什么要同事务提交?

避免回答成功但引用缺失, 或引用指向尚未完成的消息。事务失败时请求终态也回滚。

Q10: 什么是至少一次投递?

消息可能因为消费者崩溃、`ACK` 丢失等原因被重复处理, 因此消费端必须幂等。Redis Streams 归档链路属于这种语义。

Q11：如何做到归档幂等？

MongoDBStore 以 requestId 作为 key；同一事件重复投递只覆盖同一文档，写成功后再 XACK。

Q12：能否保证严格 exactly-once？

分布式系统中通常通过至少一次投递加幂等效果实现业务上的“最终只保留一份”，不要轻易宣称传输层严格 exactly-once。

Q13：事务提交后删除缓存为什么用 afterCommit？

若在事务内先删缓存后数据库回滚，缓存可能被并发请求用旧数据库值重新填充。提交后删除能让失效动作与数据库成功保持一致。

Q14：如果 afterCommit 删除 Redis 失败怎么办？

当前依靠 TTL 最终过期并记录告警；更强方案可以使用可靠 Outbox 或重试任务。

第八部分 Redis、上下文与异步归档

8.1 三类 Redis 用法

场景	数据与命令	故障行为
FAQ 缓存	String JSON、TTL、Key 索引 Set	Redis 失败回退 MySQL
验证码	随机六位码、TTL、Lua 校验并删除	失败时登录验证码不可用
Agent 上下文	checkpoint、分布式锁、Redis Stream	checkpoint 丢失尝试 Mongo 冷恢复

8.2 Cache-Aside 细节

- 读：先查 Redis，命中直接返回；未命中查已发布且分类启用的 MySQL，再写入 Redis。
- 写：先提交 MySQL 事务，afterCommit 删除分类、分页和详情缓存，后续读请求负责回填。
- 分页缓存 Key 使用查询条件规范化后 SHA-256，避免直接拼接超长或敏感参数。
- 已写 FAQ Key 登记在 Set 中，清理时不用 Redis KEYS 全库扫描。
- Redis 读取、序列化或写入失败时记录日志，但用户请求回退 MySQL。

8.3 上下文热状态与冷恢复

1. thread_id 使用 userId + sessionId，防止不同用户的相同会话主键共享上下文。
2. 运行前先查 Redis checkpointer；命中则直接恢复图状态。
3. 未命中时读取 MongoDBStore 的完整会话，显式按 created_at 与 requestId 排序。
4. 只保留最近配置数量的轮次，再通过 graph.aupdate_state 生成合法 checkpoint。
5. 在线回答结束后先 XADD 归档事件，消费者写 Mongo 成功后再 XACK。

8.4 高频问答

Q1：缓存穿透、击穿、雪崩分别是什么？

穿透是大量查询不存在数据；击穿是热点 Key 失效导致并发回源；雪崩是大量 Key 同时失效或 Redis 整体故障。可分别使用空值/布隆过滤器、互斥或逻辑过期、随机 TTL 与限流降级。

Q2：为什么不使用 Redis KEYS 清缓存？

KEYS 会遍历当前数据库，数据量大时阻塞服务。项目用 Set 登记实际缓存 Key，再批量删除。

Q3：分布式锁为什么需要过期时间？

防止持锁进程崩溃后永久死锁；但 TTL 过短会在任务未完成时释放，因此需要合理预算或续期机制。

Q4：为什么同一会话第二个请求立即失败而不是排队？

避免用户误以为两个问题都在正常执行，也避免旧回答晚于新回答写入上下文。立即失败语义更清晰。

Q5：锁释放为什么要确认 owned？

锁可能已经超时并被其他请求获取，旧持有者不能误删新锁。

Q6：Redis Stream 的 Pending Entries List 是什么？

消费者组已投递但尚未 ACK 的消息集合，记录消费者、空闲时间和投递次数，用于故障恢复。

Q7：XAUTOCLAIM 有什么作用？

健康消费者扫描空闲超过阈值的 Pending，并接管崩溃消费者遗留的消息。

Q8: 为什么 Stream 不直接 MAXLEN 裁剪?

如果裁掉仍在 Pending 的消息体, 消费组可能只剩引用而无法恢复内容。应先设计保留、积压监控与安全清理。

Q9: 为什么 Redis Search 要求逻辑库 0?

当前 LangGraph Redis Saver 需要 Redis Search 索引, 而该组件不支持在非 0 逻辑数据库创建索引, 因此配置阶段直接拒绝其他 DB。

Q10: 为什么归档不直接同步写 Mongo?

同步双写会增加聊天尾延迟并把 Mongo 故障直接传播到生成链路。Stream 解耦后可重试、接管和积压监控。

Q11: 当前归档是否已使用 RabbitMQ?

没有, 当前是 Redis Streams。RabbitMQ 只属于未来替换方向, 简历不能写成项目已经使用 RabbitMQ。

Q12: 为什么不用 MySQL chat_message 直接恢复 Agent?

业务消息可能经过展示裁剪、删除或格式变化, 且不包含图状态; Agent 的 checkpoint 与归档由独立边界维护, 语义更清晰。

第九部分 Spring Boot、MyBatis 与 MySQL

9.1 Spring 事务与 Web 基础

Q1: @Transactional 原理是什么?

Spring 通常通过 AOP 代理和 TransactionInterceptor, 在方法调用前开启或加入事务, 正常返回时提交, 匹配回滚规则的异常时回滚。

Q2: 为什么同类自调用可能导致事务失效?

this.method() 没有经过 Spring 代理, 事务拦截器无法介入。可拆分 Bean、通过代理调用, 或像项目一样显式使用 TransactionTemplate。

Q3: 默认哪些异常触发回滚?

通常 RuntimeException 和 Error; 受检异常默认不回滚, 除非配置 rollbackFor。回答时要说明具体版本和配置可能影响行为。

Q4: 事务传播 REQUIRED 是什么?

存在事务就加入, 不存在就新建, 是最常用默认传播。REQUIRES_NEW 会挂起外层并创建独立事务。

Q5: 为什么项目使用 TransactionTemplate?

流式链路包含多个明确的短事务边界和长时间外部调用, 编程式事务更容易精确控制初始化、成功、失败和取消提交范围。

Q6: Filter、Interceptor、AOP 有何区别?

Filter 属于 Servlet 容器, 适合请求级处理; Interceptor 属于 Spring MVC, 适合鉴权和 Handler 前后处理; AOP 面向 Bean 方法切面, 如事务和日志。

Q7: 全局异常处理如何实现?

通过 @RestControllerAdvice 与 @ExceptionHandler, 把参数校验、业务异常和未知异常转换为统一 Problem/错误响应, 并避免向客户端泄露内部堆栈。

Q8: 为什么要有 requestId?

串联浏览器响应、Java 日志、数据库审计和内部调用, 便于定位一次请求的完整链路。

9.2 MyBatis 高频问答

Q1: #{} 与 \${} 的区别?

#{} 通过 PreparedStatement 绑定参数, 通常更安全; \${} 是原始字符串替换, 可能 SQL 注入, 只能用于受白名单约束的动态标识符。

Q2: resultType 与 resultMap 的区别?

resultType 适合简单字段到属性映射; resultMap 可处理列名差异、嵌套对象、关联和集合, 控制力更强。

Q3: MyBatis 一级缓存是什么?

SqlSession 级缓存，同一会话重复查询可能复用结果；执行更新、提交或关闭会影响缓存。Spring 集成下应结合 SqlSession 生命周期理解。

Q4: 二级缓存是什么?

Mapper namespace 级共享缓存，需要显式配置和可序列化对象。分布式或强一致场景更常使用独立缓存并谨慎启用。

Q5: 动态 SQL 常用标签?

if、choose/when/otherwise、where、set、foreach。where/set 能自动处理多余 AND 或逗号。

Q6: 如何防止 N+1 查询?

使用 JOIN、批量 IN 查询或一次取回关联数据后在内存组装；分页时要避免 JOIN 导致主记录重复。

Q7: 为什么保留 Mapper XML?

复杂 SQL、状态条件更新和批量插入在 XML 中更直观，也便于数据库人员检查与优化。简单 SQL 可用注解，但应保持团队一致性。

9.3 MySQL 高频问答

Q1: 为什么 InnoDB 索引常用 B+Tree?

树高低、磁盘页利用率高、叶子有序，适合范围查询；内部节点只存键和指针，可以容纳更多分支。

Q2: 聚簇索引是什么?

InnoDB 主键索引叶子节点保存整行数据；二级索引叶子保存二级键和主键，查询其他列可能需要回表。

Q3: 联合索引最左前缀是什么?

索引按列顺序有序，查询通常要从最左列开始形成连续匹配；范围条件之后的列能否继续用于定位要结合执行计划。

Q4: 覆盖索引是什么?

查询需要的列都能从索引得到，无需回表，可减少随机 I/O。

Q5: MVCC 是什么?

通过隐藏版本信息、undo log 和 Read View, 让一致性读在不加普通行锁的情况下读取合适版本。

Q6: 四种隔离级别?

读未提交、读已提交、可重复读、串行化。MySQL InnoDB 默认通常是可重复读, 但面试时要说明实际配置可修改。

Q7: 幻读如何处理?

一致性读依靠 MVCC; 当前读在可重复读下可能使用 next-key lock 组合记录锁和间隙锁防止范围内插入。

Q8: 什么是死锁?

多个事务互相等待对方持有的锁形成环。应保持固定加锁顺序、缩短事务、建立合适索引, 并捕获死锁后对事务有限重试。

Q9: 为什么状态更新 SQL 要带旧状态条件?

把检查与修改合并成一次原子操作, 避免先 SELECT 再 UPDATE 中间出现竞争。

Q10: 唯一索引除了加速还有什么作用?

由数据库强制业务唯一性, 是防止并发重复插入的最后防线; 应用层先查不能替代唯一约束。

Q11: EXPLAIN 看什么?

关注访问类型、可能/实际使用索引、扫描行数估计、Extra 中的临时表和排序等, 再用真实数据验证。

Q12: 为什么不要在日志打印 SQL 参数?

账号、Token、手机号、Prompt 和正文可能包含敏感数据。项目只记录 Statement、SQL 类型、耗时和行数。

第十部分 Java 并发、虚拟线程与网络调用

10.1 项目中的并发点

- Servlet 线程完成参数校验和短事务后立即返回 SseEmitter。

- 每条内部 Agent 流由 Java 21 虚拟线程阻塞读取，不长期占用 Tomcat 请求线程。
- AtomicBoolean 记录浏览器是否仍连接，AtomicLong 生成单请求事件序号。
- AuthContext 基于 ThreadLocal，因此进入虚拟线程前必须先捕获 userId。
- 跨请求的真正竞争控制依赖数据库状态条件和 Redis 会话锁，而不是 Atomic 类型。

10.2 高频问答

Q1: 虚拟线程是什么？

JDK 管理的轻量线程，阻塞时可以卸载载体线程，使大量 I/O 等待任务共享较少平台线程。它保留直观的线程式编程模型。

Q2: 虚拟线程适合 CPU 密集任务吗？

不会凭空增加 CPU。CPU 密集任务仍受核心数限制；虚拟线程主要改善大量阻塞 I/O 的可扩展性。

Q3: 虚拟线程越多越好吗？

不是。数据库连接、下游 QPS、模型限额等资源仍需信号量、连接池或限流控制。

Q4: ThreadLocal 有什么风险？

在线程池中可能因未清理导致数据串请求；跨新线程也不会自动继承普通 ThreadLocal。项目在提交虚拟线程前先提取 userId。

Q5: AtomicLong 能保证什么？

保证单变量自增的原子性和可见性，不保证多个变量或数据库操作组成的业务事务原子性。

Q6: synchronized 与 Lock 区别？

synchronized 语法简单并由 JVM 管理；Lock 提供可中断、超时、公平锁和多个 Condition，但必须在 finally 释放。

Q7: volatile 能保证 i++ 原子吗？

不能。volatile 主要保证可见性和一定有序性；i++ 是读改写复合操作，需要 AtomicInteger 或锁。

Q8: 为什么固定 HttpClient 使用 HTTP/1.1？

Uvicorn 当前内部链路按 HTTP/1.1 工作，JDK 明文 HTTP/2 h2c Upgrade 可能造成兼容问题。内部 SSE 不依赖 HTTP/2。

Q9: 连接超时与请求超时有什么区别?

连接超时限制建立 TCP 连接; 请求超时覆盖发送请求到响应完成。SSE 请求超时必须大于正常模型生成时间。

Q10: 如何做背压?

当前链路主要靠会话锁、超时和下游限制。更大规模时需要全局并发配额、队列长度、拒绝策略和模型提供商限流。

第十一部分 认证、安全与文件治理

11.1 当前认证链路

- Access JWT 默认短期有效, 由前端通过 Authorization Bearer 发送。
- Refresh Token 使用 HttpOnly Cookie, 数据库只保存 SHA-256 摘要, 不保存明文。
- 刷新时锁定旧 Token 记录、生成新 Token 并撤销旧 Token, 实现 rotation。
- 密码使用 BCrypt; 历史 {noop} 密码在成功登录后升级。
- 短信验证码存 Java Redis, 使用 Lua 原子比较并删除, 避免同一验证码重复消费。
- 管理员与普通用户使用不同 audience/主体和路径权限。

11.2 高频问答

Q1: JWT 的优点和缺点?

无状态验证、跨服务携带方便; 但签发后难即时撤销, 载荷可解码且不应放敏感信息, 密钥轮换和过期策略也需要治理。

Q2: 为什么 Refresh Token 只存哈希?

数据库泄漏时攻击者不能直接拿记录作为登录凭证; 服务收到明文后再哈希查找。

Q3: 为什么要做 Refresh Token rotation?

每次刷新生成新 Token 并撤销旧 Token, 缩短旧 Token 被盗后的可利用窗口, 也能发现重复使用。

Q4: HttpOnly、Secure、SameSite 分别做什么？

HttpOnly 降低脚本读取风险；Secure 只通过 HTTPS；SameSite 限制跨站请求携带 Cookie，辅助防 CSRF。

Q5: BCrypt 为什么适合密码？

带随机盐且计算成本可调，能增加暴力破解成本。密码不能使用普通快速 SHA-256 直接存储。

Q6: 固定内部 Token 有什么缺点？

泄漏后可长期重放，无法验证请求时间和唯一性。生产应使用 TLS，并引入 HMAC、timestamp、nonce 与密钥轮换。

Q7: HMAC 如何防篡改？

双方基于共享密钥对规范化请求计算摘要；接收方重新计算并常量时间比较。配合时间窗口和 nonce 才能防重放。

Q8: 文件上传需要防什么？

大小限制、扩展名和内容类型校验、安全随机文件名、路径穿越、恶意文件、解析资源耗尽，以及原文件目录权限。

Q9: 为什么 objectKey 要验证 resolve 后仍在根目录？

攻击者可能传入 ../ 或绝对路径。解析规范路径后检查父目录，可以阻止越权读取服务器文件。

Q10: 日志为什么不能记录请求体？

请求体可能包含密码、Token、手机号、用户问题和企业文档内容。应记录 requestId、状态、耗时和必要的脱敏标识。

第十二部分 Python、FastAPI 与异步编程

12.1 高频问答

Q1: async/await 适合什么？

适合大量 I/O 等待，例如模型流、HTTP 请求和 Redis。CPU 密集计算不会因为 async 自动并行，应使用进程池、原生库或任务系统。

Q2: Python GIL 是什么?

CPython 中保护解释器对象状态的全局锁, 使同一进程内多个线程通常不能并行执行 Python 字节码; I/O 等待和释放 GIL 的原生扩展不完全受此限制。

Q3: 为什么 Chroma 同步操作放 `asyncio.to_thread`?

避免阻塞事件循环, 使 FastAPI 仍能处理其他 I/O 请求。它适用于阻塞 I/O 或会释放 GIL 的库, 不等于无限扩容。

Q4: FastAPI StreamingResponse 如何工作?

接收同步或异步迭代器, 迭代器每次 yield 一段字节/字符串并写入响应。SSE 还需设置 `text/event-stream` 和正确事件格式。

Q5: Pydantic 的价值?

在接口边界完成类型转换、字段校验和稳定错误响应, 使 Java camelCase DTO 与 Python 模型契约可测试。

Q6: 依赖注入在 FastAPI 中做什么?

把配置、鉴权和运行时对象从路由逻辑中分离, 便于测试替换和统一校验。

Q7: 为什么捕获异常后不能全部返回 500?

调用方需要区分取消、上下文存储不可用、RAG 降级和模型故障, 才能决定是否重试以及如何提示用户。

Q8: 为什么测试模式使用 InMemorySaver 和空归档?

单元测试不应访问开发者真实 Redis/Mongo; 通过最小协议替换外部依赖, 测试可以稳定验证流事件与图分支。

Q9: asyncio.Lock 与 Redis 锁有什么区别?

`asyncio.Lock` 只保护单进程事件循环内协程; Redis 锁可协调多个进程或实例。当前 RAG 索引同步用进程内锁, 会话运行用 Redis 锁。

Q10: 配置为什么使用环境变量和 Secret 类型?

避免把密钥写进代码与版本库; Secret 类型还能降低被日志或 repr 意外打印的风险。

Q11: FastAPI 的 `async def` 和 `def` 路由有什么差别?

`async def` 在事件循环中执行，内部应使用非阻塞 I/O；普通 `def` 会进入线程池，适合无法异步化的阻塞调用。把阻塞代码直接放进 `async def` 会卡住事件循环。

Q12: Pydantic 校验能替代业务校验吗?

不能。Pydantic 适合类型、范围、格式和字段关系；资源归属、状态迁移、权限与数据库唯一性仍属于业务和持久层约束。

Q13: 依赖注入如何支持测试?

路由依赖抽象的配置、鉴权主体和服务对象，测试时通过 `dependency_overrides` 替换为内存实现或 `stub`，避免连接真实模型、Redis 和向量库。

Q14: BackgroundTasks 适合做可靠任务吗?

不适合关键可靠任务。它与 Web 进程生命周期绑定，进程退出会丢失。轻量通知可用；归档、批量 Embedding 等需要队列、持久状态与重试。

Q15: Uvicorn 多 worker 会带来什么问题?

每个 worker 是独立进程，内存锁、取消注册表和缓存互不共享。多实例下要把协调状态迁移到 Redis 或数据库，并确保向量库写入策略可并发。

Q16: 接口返回 422、400、409、503 怎么区分?

422 常表示请求结构校验失败；400 是语义不合法；409 表示状态冲突或重复操作；503 表示暂时不可用且可能重试。具体契约需团队统一，不能随意混用。

Q17: 生成器被客户端断开后如何清理?

在异步生成器的 `try/finally` 中释放下游连接、锁和观测资源，并区分网络断开与业务取消。必要时屏蔽取消以完成关键终态写入，但不能无限拖延。

Q18: Python 类型标注在项目中有何价值?

它让 Pydantic 模型、事件联合、服务协议和返回类型更清晰，配合静态检查减少跨模块错误；但它默认不做运行时强制，外部输入仍需验证。

第十三部分 项目边界、风险话术与简历修改

13.1 当前真实边界

主题	当前实现	正确面试话术
分块	递归字符分块	可配置 chunk/overlap; 尚未做语义分块
检索	稠密向量 TopK + 阈值	尚未加入 BM25、混合检索和 Reranker
RAG 评测	以单元测试验证流程	还需要真实问答集与离线指标
取消	Python 进程内取消注册表	单实例可用; 多实例需 Redis 等共享状态
异步归档	Redis Streams	RabbitMQ 尚未接入
内部鉴权	固定 X-Internal-Token	生产需 TLS + HMAC + nonce
Chat 幂等	状态更新和取消幂等	通用 Idempotency-Key 尚未落地
高并发	虚拟线程、会话锁、异步归档	未完成正式压测, 不声称具体 QPS
React	没有 React 项目实现	可讲 Vue 到 Hooks 的概念映射, 不声称上线经验
Node 后端	仅用于 Vite/TypeScript 工具链	了解事件循环和 BFF; 业务服务实际为 Java/Python
Markdown	回答当前按纯文本安全呈现	如引入 Markdown, 必须增加解析、XSS 清洗和链接治理

成熟回答：承认边界不会减分。先说明当前选择满足什么规模，再给出可落地的演进方案，比笼统声称“高可用、高并发、智能分块”更可信。

13.2 简历必须修改的问题

- 删除第二页大面积空白，把自我评价压缩到第一页或直接删除，尽量形成一页校招简历。
- MyMatis 改为 MyBatis; Springboot 改为 Spring Boot; Langchain 改为 LangChain。
- “智能分块”改为“可配置递归字符分块”，除非后续确实实现语义分块。
- 没有独立 RabbitMQ 实战时，把“有一定使用经验”改成“了解消息确认、重复消费和失败重试；项目使用 Redis Streams”。
- 技能栏减少重复的“熟悉/了解”，用实际项目证据替代抽象自评。
- 个人职责只写自己能现场解释代码、数据流、失败分支和权衡的部分。

- 为该岗位把技能顺序调整为：JavaScript/TypeScript/Vue → Python/FastAPI/LangChain → API 与 Java/Spring。
- 若能现场解释前端代码，在项目经历中补充流式解析、Pinia 状态机、AbortController 和 401 单飞刷新。
- React 和 Node.js 后端没有项目证据时写“了解核心机制”，不要与 Vue、Python 写成同一熟练度。

13.3 推荐的项目经历写法

版本 A：严格贴合 AI / RAG 职责：基于 FastAPI、LangChain 与 LangGraph 实现受控问答 workflow，完成输入处理、DIRECT/RAG 路由、查询改写、向量检索、流式生成及异常降级；负责维修手册 RAG 链路，支持 PDF/DOCX/TXT/MD 解析、递归字符分块、Embedding、Chroma 持久化与引用元数据返回；参与 Spring Boot、MyBatis、MySQL 后台管理接口开发。

版本 B：确认能解释前端代码后优先用于本岗位：在版本 A 基础上补充：使用 Vue 3、TypeScript 与 Pinia 实现智能客服交互，基于 fetch/ReadableStream 解析 POST SSE 的状态、增量文本、引用和终态事件；通过 AbortController、requestId 终态查询与 401 单飞刷新处理停止生成、断线恢复和并发鉴权，并参与 Java/Python API 契约与聊天终态设计。

13.4 不要编造的指标

- 没有压测报告，不写“支持上千并发”或具体 QPS。
- 没有标注问答集，不写“准确率提升 30%”。
- 没有成本统计，不写“Token 成本降低 50%”。
- 可以写可验证事实：支持的文件类型、图节点、索引指纹、事件类型、缓存模式、测试场景。

第十四部分 面试流程、答题方法与模拟题

14.1 常见面试推进方式

1. 自我介绍与项目概览：通常 3-5 分钟，重点看表达是否清晰、职责是否可信。

- 2. 前端基础：HTML/CSS 布局、JavaScript 事件循环、Promise、Vue 响应式、状态管理与工程化。
- 3. LLM 项目纵向深挖：从聊天交互追问到 SSE、FastAPI、LangGraph、RAG 和异常降级。
- 4. API 集成：鉴权刷新、幂等、错误契约、requestId、取消与断线终态恢复。
- 5. 后端加分项：Java 并发、Spring 事务、MyBatis、MySQL 索引、MVCC 和 Redis。
- 6. 场景设计：流式渲染性能、XSS、模型超时、索引更新、重复请求和权限隔离。
- 7. 反问环节：询问团队 AI 应用落地、工程栈、评测方式和新人培养，不只问加班。

14.2 四段式答题法

步骤	要回答什么	示例
为什么	业务问题与约束	模型调用慢，不能持有长事务
怎么做	具体代码和数据流	初始化短事务 + 外部调用 + 终态事务
保证什么	正确性或收益	减少连接占用，避免半条请求
代价/边界	没有解决什么	跨服务无法用本地事务，需要状态机与恢复

14.3 七个必须准备的 STAR 故事

- 前端响应式问题：流式内容已更新但页面不刷新，如何定位普通对象引用绕过 Proxy，并改为先 reactive 后入列。
- 鉴权并发：多个请求同时 401 时，如何用共享 refreshPromise 合并刷新，避免 Token rotation 竞争。
- 索引一致性：为什么不能让 Java 和 Chroma 双写，最终如何用清单与 fingerprint 解决。
- 知识库降级：为什么清单失败时禁止查询旧索引，如何给用户稳定提示。
- 取消竞争：取消和成功并发时，为什么依靠数据库状态条件而不是内存标记。
- 上下文恢复：Redis checkpoint 丢失后如何按顺序从 Mongo 重建最近轮次。
- 流式故障：浏览器断线、Python 中断和数据库提交失败分别如何处理。

14.4 模拟连环追问

面试官：你为什么选择 LangGraph？

先回答存在条件分支、checkpoint 和节点降级；再说明当前是确定性图，不是为了炫技；最后补充简单链路用普通 Chain 更合适。

面试官：向量库里有旧版本怎么办？

说明 MySQL 与原文件是真相源，以 documentId 分组比较 fingerprint；变化先删旧切片再写新切片；清单失败本轮不查旧索引。

面试官：如果删除旧切片后 Embedding 失败呢？

当前会导致该文档暂时没有切片并让检索降级。更强方案是新版本写入临时集合/版本命名空间，全部成功后再原子切换。

面试官：同一会话同时问两个问题？

Redis 分布式锁以 userId+sessionId 为粒度，第二个请求立即失败；不同会话可并发。这样保护 checkpoint 消息顺序。

面试官：Redis Stream 消费者写 Mongo 后 ACK 前崩溃？

消息会留在 Pending，之后被再次处理；requestId 作为存储 key 使重复写覆盖同一文档，最终再 ACK。

面试官：前端收到了部分回答后断网？

服务端不把断网自动当取消，继续完成并落库；前端恢复后用 requestId 查询终态。用户明确停止则另调取消接口。

面试官：你的 RAG 准确率是多少？

如果没有正式评测，不报虚假数字。说明当前参数、已有流程测试，并提出构建标注集评 Recall@K、忠实度、引用正确率和拒答率。

面试官：为什么不用 RabbitMQ？

当前已有 Redis, Streams 足以支持消费组、ACK 与接管，降低部署成本；复杂路由、死信治理和独立消息平台需求增强后再迁移。

面试官：为什么不用 EventSource?

需要 POST 请求体、Authorization 和 Idempotency-Key，而 EventSource 对方法和自定义头限制较多，因此使用 fetch 读取 text/event-stream。

面试官：Vue 流式消息为什么要 reactive?

后续 delta 会原位修改消息对象；如果持续修改的是 push 前普通对象引用，可能绕过数组内部代理。先创建 reactive 对象并保存同一代理引用可以稳定触发视图更新。

面试官：你会 React 吗?

项目主栈是 Vue 3，没有把 React 包装成上线经验；但能把 ref/computed/watch、Pinia 和组件通信映射到 state、memo、effect 与状态容器，并能快速迁移。

面试官：你会 Node.js 吗?

当前 Node 用于 Vite/TypeScript 构建，不是业务后端。我掌握事件循环、非阻塞 I/O、中间件和 BFF 场景；本项目已有 Java 统一入口，因此没有为了凑栈再加 Node 服务。

14.5 反问面试官

- 团队目前 AI 应用的主要落地方向是什么：知识库、Agent、流程自动化还是平台工程？
- 团队如何评测 RAG 或 Agent 的效果，是否有离线集、线上反馈和可观测平台？
- 这个岗位的前端、Python RAG 和 API 集成工作量大致如何分配？主前端技术栈是 Vue 还是 React？
- LLM 流式交互、引用展示和评测平台目前由哪一层团队负责？
- Java、Node 与 Python 在实际项目中的服务边界如何划分？
- 新人入职前三个月通常负责什么类型的任务，代码评审和导师机制如何？
- Direct Pass 的考核周期和核心评价维度是什么：交付质量、学习速度、业务理解还是独立负责能力？

第十五部分 14 天复习计划与自测清单

15.1 时间分配

推荐比例：前端与 JavaScript/Vue 35%，Python/FastAPI/RAG 35%，API 集成与完整项目链路 20%，Java/Spring/MySQL/Redis 10%。岗位虽然接受多种技术栈，但你的优势应是能从浏览器交互一直讲到向量检索和服务终态。

天数	主题	产出
第 1 天	JD 拆解；背熟 30 秒、90 秒、3 分钟介绍	录音 3 次，主线对准前端 + RAG + API
第 2 天	HTML 语义、盒模型、Flex/Grid、响应式	不看代码解释聊天页布局
第 3 天	JavaScript 作用域、闭包、this、原型	手写并口述基础例子
第 4 天	事件循环、Promise、async/await、Fetch	画任务与微任务顺序
第 5 天	Vue 3 响应式、computed/watch、生命周期	解释 reactive 消息修复
第 6 天	Pinia、Router、组件通信、TypeScript	从组件讲到 service/store 分层
第 7 天	POST SSE、buffer、Abort、断线恢复	手写事件解析和状态机
第 8 天	React Hooks 对照与 Node.js 事件循环	诚实讲迁移能力和项目边界
第 9 天	Python async、FastAPI、Pydantic、依赖注入	回答第十二部分全部问题
第 10 天	LangGraph、RAG 索引与在线检索	画两条数据流水线
第 11 天	Embedding、Chroma、评测、降级	解释参数、边界和演进
第 12 天	API 契约、JWT 刷新、幂等、requestId	画跨服务成功和失败链路
第 13 天	完整模拟面试	按问题记录卡顿点
第 14 天	Java/Spring/MySQL 快速复盘并修订简历	保留 Java 加分项，只写能解释的技能

15.2 上场前自测

- 能在 90 秒内围绕前端交互、Python RAG 和 API 集成讲清业务、职责与难点。
- 能区分项目整体实现与自己的真实贡献。
- 能解释语义化 HTML、盒模型、Flex/Grid、CSS 优先级和响应式布局。
- 能解释闭包、this、原型链、事件循环、Promise 与 async/await。
- 能解释 Vue ref/reactive、computed/watch、nextTick、key 和生命周期。

- 能解释为什么流式消息要先 reactive 再放入 Pinia 数组。
- 能把 Vue 概念映射到 React Hooks，同时明确没有 React 上线经验。
- 能解释 Node.js 事件循环、中间件与 BFF，并明确项目未使用 Node 后端。
- 能不看代码画出 input_guard 到 generate 的图。
- 能解释 fingerprint、chunk ID、TopK、阈值和引用元数据。
- 能解释为什么清单失败时不查询旧 Chroma。
- 能解释 fetch SSE 的 buffer、sequence 和 terminal 校验。
- 能解释 AbortController 与服务端取消接口为什么必须同时存在。
- 能解释 401 single-flight 刷新、幂等键、requestId 和错误契约。
- 能说明安全渲染 Markdown 所需的解析与 XSS 清洗措施。
- 能解释为什么数据库提交后才向浏览器发送 done。
- 能用时间线解释取消和成功竞争。
- 能解释会话锁、锁过期和 owned 检查。
- 能解释 Redis Stream Pending、XACK 和 XAUTOCLAIM。
- 能解释 Cache-Aside 和 afterCommit 删除缓存。
- 能解释 Spring 事务代理、自调用和短事务边界。
- 能解释 MyBatis #{}, \${}、resultMap 和动态 SQL。
- 能解释联合索引、MVCC、隔离级别和 next-key lock。
- 能解释虚拟线程为什么适合 SSE 阻塞 I/O。
- 能明确说出项目没有实现的能力，不编造指标。

附录 高质量资料入口与代码复习路径

A.1 中文八股与系统学习入口

MDN Web 开发学习: https://developer.mozilla.org/zh-CN/docs/Learn_web_development。按 HTML、CSS、JavaScript、可访问性和 Web API 建立可靠基础。

现代 JavaScript 教程: <https://zh.javascript.info/>。系统学习语言基础、Promise、事件循环、DOM 与浏览器 API。

Vue 3 中文文档: <https://cn.vuejs.org/guide/introduction.html>。重点复习响应式、组件、计算属性、侦听器、生命周期和组合式函数。

前端面试问题集: <https://github.com/h5bp/Front-end-Developer-Interview-Questions>。用于查漏补缺；答案应回到 MDN 与框架官方文档验证。

JavaGuide: <https://github.com/Snailclimb/JavaGuide>。用于建立 Java、JVM、Spring、MySQL、Redis、分布式和系统设计目录。

小林 Coding / CS-Base: <https://github.com/xiaolincoder/CS-Base>。重点补网络、操作系统、MySQL 索引与事务、Redis。

A.2 官方文档

- MDN JavaScript Event Loop - 任务、微任务与运行到完成：
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Event_loop
- MDN Fetch API - 请求、Response、ReadableStream 与异常处理：
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN AbortController - 中止 fetch 与流读取：
<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>
- Vue Reactivity Fundamentals - ref、reactive、DOM 更新时机：
<https://vuejs.org/guide/essentials/reactivity-fundamentals.html>

- Pinia Core Concepts - State、Getter、Action 与 setup store:
<https://pinia.vuejs.org/core-concepts/>
- React Learn - State、组件、Effect 与 Hooks: <https://react.dev/learn>
- Node.js Event Loop - 事件循环、timers 与 nextTick:
<https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>
- TypeScript Handbook - 类型系统、缩小与泛型:
<https://www.typescriptlang.org/docs/handbook/intro.html>
- LangGraph Persistence - State、thread_id、Checkpoint 与 Store:
<https://docs.langchain.com/oss/python/langgraph/persistence>
- LangChain Retrieval - 加载、分块、Embedding、Vector Store 与 Retriever:
<https://docs.langchain.com/oss/python/langchain/retrieval>
- Chroma Query and Get - 向量查询、TopK、metadata filter:
<https://docs.trychroma.com/docs/querying-collections/query-and-get>
- Spring Transaction Management - 事务抽象、声明式和编程式事务:
<https://docs.spring.io/spring-framework/reference/data-access/transaction.html>
- MyBatis Mapper XML - #{}, \${}、resultMap 与 Mapper XML:
<https://mybatis.org/mybatis-3/sqlmap-xml.html>
- MySQL InnoDB Locking - 隔离级别、锁、幻读和死锁:
<https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-transaction-model.html>
- Redis Cache-Aside - 缓存读写与失效模式: <https://redis.io/docs/latest/develop/use-cases/cache-aside/>
- Redis XAUTOCLAIM - Pending 消息故障接管:
<https://redis.io/docs/latest/commands/xautoclaim/>
- MDN Server-Sent Events - SSE 协议基础: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- FastAPI Stream Data - StreamingResponse:
<https://fastapi.tiangolo.com/advanced/stream-data/>
- OpenJDK JEP 444 - Java 21 虚拟线程: <https://openjdk.org/jeps/444>

A.3 当前项目代码复习顺序

1. frontend/src/services/chat.ts: fetch、ReadableStream、TextDecoder、SSE buffer 与终态校验。
2. frontend/src/stores/chat.ts: Pinia 状态、reactive 消息、AbortController、错误恢复与反馈。
3. frontend/src/pages/ChatPage.vue: computed、watch、nextTick、交互状态与响应式布局。
4. frontend/src/services/http.ts: Axios 拦截器、401 single-flight 刷新、requestId 与 Problem Details。
5. frontend/src/components/ChatMessage.vue: props/emits、稳定 key、引用呈现与文本安全边界。
6. agent_service/src/agent_service/graph/workflow.py: 图节点、路由、检索、生成与降级。
7. agent_service/src/agent_service/graph/state.py: State 与 Runtime Context。
8. agent_service/src/agent_service/services/manual_rag.py: 清单、解析、分块、Chroma。
9. agent_service/src/agent_service/services/agent_runtime.py: thread_id、会话锁、冷恢复与流执行。
10. agent_service/src/agent_service/services/context_archive.py: Stream 发布与 Mongo 恢复。
11. agent_service/src/agent_service/workers/context_archive_worker.py: 消费组、ACK 与接管。
12. xc_agent/src/main/java/com/xc/agent/service/impl/ChatServiceImpl.java: 短事务、状态机、SSE 终态。
13. xc_agent/src/main/java/com/xc/agent/service/impl/InternalAiServiceImpl.java: JDK HttpClient 与内部 SSE 解析。
14. xc_agent/src/main/java/com/xc/agent/service/content/FaqCacheService.java: Cache-Aside 与 afterCommit。
15. database/mysql/01_schema.sql: 表、索引、约束与状态字段。

A.4 最终提醒

面试原则：能解释代码和权衡的内容才写“熟悉”；只看过概念写“了解”；项目尚未实现的能力用“演进方向”表达。真实、具体、可追问，比堆砌名词更有竞争力。

— 完 —